

INTERNSHIP PROJECT REPORT

Pearls AQI Predictor

An End-to-End Serverless MLOps Pipeline for 72-Hour Air Quality Forecasting

Prepared for: 10Pearls Software Engineering Internship

Muhammad Hashir Awaiz

BS Artificial Intelligence

Ghulam Ishaq Khan Institute of Engineering Sciences and Technology (GIKI)

September 2026

Live Application: aqi-predictor-sialkot-data-science-project.streamlit.app
Model Registry: huggingface.co/HashirAwaiz/aqi-forecast-model

1. Executive Summary & Problem Statement

1.1 Problem Statement

Air quality in Sialkot, like much of industrial Punjab, is subject to significant day-to-day and seasonal volatility, driven by a mix of industrial emissions, vehicular traffic, and regional weather patterns. Reactive, same-day AQI reporting gives residents and public-health stakeholders little time to plan around unhealthy air. The core problem this project addresses is the absence of a forward-looking, reliable signal: a system that can tell a resident not just what the air quality is right now, but what it is likely to become over the next several days, with enough lead time to take protective action.

The objective set for this internship project was to design and build a forecasting system capable of predicting Sialkot's Air Quality Index 72 hours (three days) in advance, and to do so using a production-oriented engineering approach rather than a one-off notebook analysis.

1.2 Objective

The stated objective was to build a 100% serverless, end-to-end MLOps pipeline: automated data ingestion, a persistent feature store, chronological model evaluation across multiple algorithm families, automated retraining and model registration, explainability reporting, and a live public-facing dashboard all running on scheduled automation rather than an always-on server.

- **Forecast horizon:** 72 hours (3 days) ahead.
- **Deployment model:** fully serverless, free-tier infrastructure, orchestrated by scheduled CI/CD jobs rather than a persistently running application server.
- **Deliverable:** a live Streamlit dashboard backed by an automated, self-retraining ML pipeline.

2. Technology Stack & Infrastructure

Every component in the stack was chosen to satisfy two constraints simultaneously: it had to support a scheduled, non-interactive automation workflow, and it had to be operable on a free tier without requiring a persistent server or, in practice, a payment card. That second constraint ended up shaping the architecture as much as any technical requirement.

Category	Technology
Language	Python 3.12
Data Ingestion	OpenWeather API, Open-Meteo API
Feature Store / Database	Supabase (PostgreSQL)
Machine Learning	scikit-learn, XGBoost, LightGBM, PyTorch
Explainability	SHAP
Model Registry	Hugging Face Hub
CI/CD Orchestration	GitHub Actions

Category	Technology
Dashboard	Streamlit
Deployment	Streamlit Community Cloud

2.1 Infrastructure Decisions

- **Feature store Supabase over Hopsworks / Vertex AI:** the project initially targeted Hopsworks as the feature store and model registry, in line with common MLOps reference architectures. A platform-side issue blocked free-tier project creation, so the pipeline was migrated to Google Vertex AI with BigQuery as the feature store. Vertex AI's free trial required a linked billing account, and ultimately neither Hopsworks nor Vertex AI could be provisioned without a payment card. The pipeline was re-architected around Supabase (PostgreSQL) for the feature store, which offers a genuinely free tier with transaction pooling and no card requirement a more pragmatic fit for a student-run, always-on cron pipeline.
- **Model registry Hugging Face Hub:** chosen alongside Supabase for the same reason free-tier storage for versioned model artifacts (trained model, scaler, SHAP output) without infrastructure overhead.
- **Orchestration GitHub Actions:** rather than Apache Airflow or a managed workflow engine, GitHub Actions' cron-triggered workflows were used to run the feature pipeline hourly and the training pipeline daily, keeping the entire system serverless.

3. Data Engineering & Feature Pipeline

3.1 Data Sources

The pipeline draws on two external APIs with complementary roles:

- **OpenWeather API:** supplies live and historical pollutant concentration data.
- **Open-Meteo API:** supplies historical weather data used to enrich the pollutant signal with meteorological context.

3.2 Feature Engineering

Each hourly run of the feature pipeline computes 42 model-ready features from the raw pollutant and weather readings before upserting them into the Supabase feature store. The feature set includes cyclical time encodings (sine/cosine transforms of hour-of-day and time-of-year, so the model can represent daily and seasonal periodicity without an artificial discontinuity at midnight or year-end) together with derived variables such as short-term AQI change rate, alongside the raw pollutant and weather readings themselves.

3.3 Historical Backfill

To give the training pipeline enough history to learn from on day one, a 90-day historical backfill merges OpenWeather's pollutant history with Open-Meteo's weather history into the Supabase feature store, with deduplication handling so repeated backfill runs do not corrupt the table.

3.4 Chronological Data Flow

Once populated, the feature store serves two consumers: the training pipeline, which pulls the full historical table to build and evaluate models, and the Streamlit dashboard, which reads only the most recent row to generate a live prediction.

4. Model Training, Evaluation & Selection

4.1 Chronological Train/Test Split

Time-series forecasting is especially vulnerable to data leakage under a naive random train/test split, since a randomly sampled test point can sit chronologically before training points the model has already seen implicitly giving the model access to the future. This pipeline instead uses a strict chronological split: all training observations precede all test observations in time, so the evaluation genuinely measures the pipeline's ability to predict unseen future conditions from past data, mirroring how the system will actually be used in production.

On the most recent training run, the feature store contained 2,041 rows. After constructing the 72-hour-ahead target (each row paired with the AQI recorded 72 hours later), 1,911 rows had a valid label. The chronological split produced 1,528 training rows and 383 test rows, across 42 features.

4.2 Models Evaluated

Five candidate model families were trained and evaluated against a naive persistence baseline, which simply predicts that the AQI 72 hours from now will equal the AQI right now a standard reference point for judging whether a learned model adds real forecasting value:

- **XGBoost** gradient-boosted trees, chosen to capture non-linear feature interactions.
- **LightGBM** a faster gradient-boosting implementation, evaluated for efficiency and accuracy on tabular data.
- **Random Forest** an ensemble of randomized decision trees, robust to unscaled features and non-linear relationships.
- **Ridge Regression** a regularized linear model, used to establish how much signal is captured by roughly linear relationships alone.
- **PyTorch MLP** a small neural network, evaluated to test whether a learned non-linear representation could outperform tree-based methods.

4.3 Results

Model	RMSE	MAE	R ²
-------	------	-----	----------------

Model	RMSE	MAE	R ²
Persistence Baseline	51.08	33.00	0.028
XGBoost	43.49	32.66	0.657
LightGBM	35.52	24.98	0.106
Random Forest (selected)	33.34	23.02	0.71
Ridge Regression	45.20	35.54	0.791
PyTorch MLP	40.63	28.61	0.447

The pipeline selects the production model purely on lowest test-set RMSE, which Random Forest achieved at 33.34 a 34.7% reduction versus the persistence baseline's 51.08, alongside a MAE reduction from 33.00 to 23.02 (30.2% lower).

4.4 Interpreting R² = 0.71

An R² of 0.71 means the Random Forest model explains roughly 71% of the variance in held-out AQI values relative to a mean-prediction reference a strong result for a 72-hour-ahead environmental forecasting task, where conditions can shift substantially over three days. R² is read alongside RMSE and MAE rather than in isolation, since a model can post a reasonable R² while still carrying an error magnitude that matters practically; here, both the variance-explained figure and the absolute error reduction versus the baseline point in the same direction.

4.5 Selected Model

Random Forest was pushed to production: `model.joblib`, its accompanying `scaler.joblib`, and a `shap_summary.png` explainability artifact were uploaded to the Hugging Face repository HashirAwaiz/aqi-forecast-model, from which the live dashboard loads them for inference.

5. Challenges Faced & Engineering Solutions

5.1 Challenge: Free-Tier Infrastructure Kept Failing

The project's first architecture targeted Hopworks for the feature store and model registry, in line with typical MLOps reference designs. A platform-side bug prevented free-tier project creation. The pipeline was re-pointed at Google Vertex AI with BigQuery, but Vertex AI's free trial required a linked billing account, and like Hopworks ultimately could not be provisioned without a payment card.

- **Solution:** the feature store was moved to Supabase (PostgreSQL, free tier, no card required) and the model registry to Hugging Face Hub (also free, no card), which together restored a genuinely free, always-available infrastructure path and unblocked the rest of the pipeline.

5.2 Challenge: Environment and Dependency Conflicts

Hopworks' `hsfs` client constrained the project to Python <3.13, ruling out a newer interpreter version. Separately, installing the `pyjks` dependency chain on Windows failed on the `twofish` package for lack of a C compiler toolchain.

- **Solution:** the project was pinned to a Python 3.12 virtual environment, and the Visual C++ Build Tools (Desktop development with C++ workload) were installed to satisfy the native build requirement.

5.3 Challenge: Duplicate-Key Errors During Historical Backfill

Backfilling historical weather and pollutant data in chunks produced overlapping date ranges at chunk boundaries, which surfaced as duplicate primary-key errors when writing to Supabase.

- **Solution:** the backfill script was updated with deduplication logic ahead of the database write and an upsert (insert-or-update) strategy, so overlapping boundary rows are reconciled rather than rejected.

5.4 Challenge: All Models Returned a Negative R^2

On the first full training run against the 72-hour-ahead AQI target, every trained model across all three algorithms evaluated at that stage produced a negative R^2 , meaning each was performing worse than simply predicting the mean of the target. This is a common but easy-to-miss failure mode in early time-series pipelines, since a negative R^2 on its own does not say whether the problem is the features, the model, or the target definition.

- **Solution:** a naive persistence baseline (predict that AQI 72 hours from now equals AQI now) was added directly into the evaluation pipeline as a diagnostic reference point, so every future run could immediately answer "is the model actually learning something beyond the naive guess?" rather than relying on R^2 in isolation. Combined with continued iteration on the feature set expanding to the current 42 engineered features, including cyclical time encodings and derived pollutant/weather variables this diagnostic-first approach was what ultimately drove Random Forest to a positive R^2 of 0.71, comfortably ahead of the persistence baseline on every metric.

5.5 Challenge: Orchestrating CI/CD Without an Always-On Server

A conventional MLOps deployment would run scheduled jobs from a persistent worker or managed workflow engine such as Apache Airflow infrastructure this project deliberately avoided in order to stay serverless and free-tier.

- **Solution:** GitHub Actions cron workflows were used instead: an hourly workflow (15 * * * *) runs the feature pipeline, and a daily workflow (0 12 * * *, 17:00 PKT) runs the full training and model-registration pipeline. Both run on GitHub's hosted runners, so no infrastructure is owned or maintained between runs.

6. Web Dashboard & Real-Time Inference

The public-facing component of the project is a Streamlit application that turns the trained model into a live, usable forecast rather than a static evaluation report.

On each page load, the dashboard performs three steps in sequence:

- Loads the current production model and its preprocessing scaler the latest versioned artifacts from the Hugging Face model registry.
- Queries Supabase for the single most recent row in the feature store, representing current pollutant and weather conditions.

- Runs inference to produce a 72-hour-ahead AQI prediction, then classifies that value against health-advisory thresholds and surfaces the result as an actionable alert alongside current conditions.

This separation a slow, scheduled training loop feeding a fast, stateless inference loop is what allows the dashboard to stay responsive and cheap to run while the underlying model quietly improves in the background every day.

7. Key Learnings & Skill Acquisition

Beyond the AQI forecasting problem itself, this project was a working exercise in end-to-end MLOps under real infrastructure constraints. Key takeaways:

- **End-to-end pipeline ownership:** designing and operating every stage from raw API ingestion through feature engineering, training, model registration, and serving, rather than working in an isolated notebook on a static dataset.
- **Infrastructure trade-off analysis:** evaluating and re-architecting around three different feature-store/registry combinations (Hopworks, Vertex AI + BigQuery, and finally Supabase + Hugging Face) under real free-tier and provisioning constraints, rather than assuming an initial technology choice is final.
- **Diagnosing model failure honestly:** treating a negative R^2 as a signal to instrument the pipeline with a baseline comparison rather than immediately reaching for a more complex model a diagnostic-first habit that generalizes well beyond this project.
- **Time-series-specific rigor:** internalizing why a chronological train/test split matters for forecasting problems, and why a naive persistence baseline is the correct bar for judging whether a model earns its complexity.
- **Automated, scheduled systems:** building and debugging CI/CD workflows (GitHub Actions) that must run unattended on a fixed schedule, which surfaces a different class of bug than interactive development idempotency, deduplication, and upsert logic became necessary rather than optional.
- **Model explainability:** generating and shipping SHAP-based feature-importance artifacts alongside the model itself, so predictions remain interpretable in production rather than being a black box.
- **Production-grade Python:** structuring the codebase into discrete, single-responsibility pipeline scripts (feature pipeline, backfill, training, EDA, dashboard) rather than a single monolithic notebook, in preparation for scheduled, unattended execution.

Repository: github.com/Muhammad-Hashir-55/aqi-predictor | Live app: aqi-predictor-sialkot-data-science-project.streamlit.app | Model registry: huggingface.co/HashirAwaiz/aqi-forecast-model